Safe Fruit

从回溯到森林动态规划

算法设计与分析 · Project 3

版本演进、消融对比与性能分析

组长　游龙斌　3250106392

组员　陈智锟　3250100883

组员　黄绎睿　3250104426

2026.10

Safe Fruit：水果选取

从一篮水果中，选取可以安全地一起食用的水果。

水果种类	用三位编号来表示水果种类，如 001、002。
数据大小	输入的水果最多 100 种，选取规则最多 100 条。
输入数据	一是选取规则，一条规则里的两种水果不能同时选取。 二是每种水果的价格，价格是不超过 1000 的正整数。

选取的最终目标是选取尽可能多的水果；在数量相同时，选取总价格尽量小的方案。

数据存储与回溯思路

数据存储

水果看作节点，不能同时选取的规则看作边。

用二维数组记录每种水果的冲突邻居，用一维数组记录价格。

选取思路

采取回溯法：每种水果都有“选”和“不选”两种情况。

选：记录水果，并排除冲突水果

不选：继续讨论剩余水果

不进行任何优化，100 种水果需要讨论 2^100 种情况；主要工作放在剪枝上。

把“数量优先、价格次之”合成一个分数

每种水果先给一个足够大的基础分，再根据价格扣掉一小部分。

每种水果的分数：$w(i) = 100001 - p(i)$

为什么取 100001?

水果最多 100 种，每种价格最多 1000：全部价格之和最多 100000。

100001 比总价格可能产生的差额更大，多选一种的增益一定占优。

同样数量下，价格越低，分数越高；后续只需比较一个总分。

共有的基础优化

以下三项用于减少搜索工作，贯穿各版本。

01 **分数上界剪枝**

当前得分 + 剩余候选的最高可能得分 ≤ 已知最优分

当前分支无法改进答案，提前终止。代码用冲突配对收紧这个上界。

02 **掩码保存状态**

一个候选集合用两个 64 位整数表示，覆盖最多 100 种水果。

按位操作即可检查、加入或移除水果，快速更新搜索状态。

03 **先选无冲突水果**

没有任何冲突的水果一定可以选入答案，因为它能增加水果数量。

程序先把它们选上，再从回溯搜索中移除。

这些优化由各版本共用，不作为版本差异单独比较。

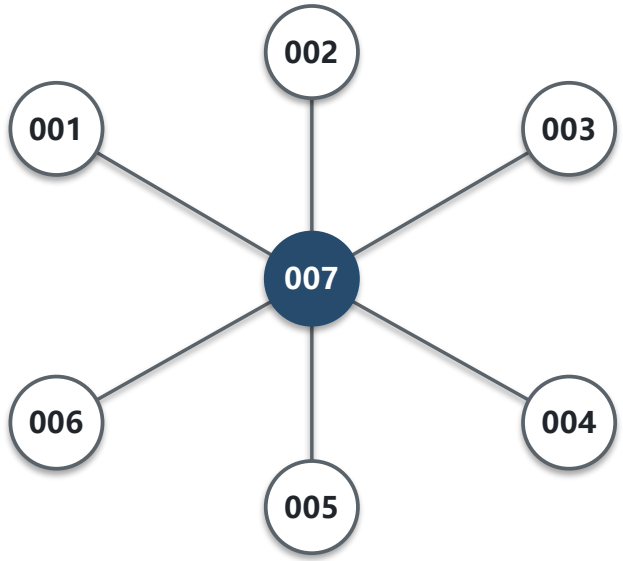
版本设计：按优化目标组织对比

版本	分支选点	分量分解	新增结构处理
v1	固定顺序	否	基础回溯
v2	动态最大候选度	是	作为后续基线
v3	固定顺序	是	与 v2 对比选点
v4	动态最大候选度	否	与 v2 对比分量
v5	初始度数预排序	否	静态顺序
v6	沿用 v2	是	原始树分量 DP
v7	沿用 v2	是	回溯中检测森林
v8	沿用 v2	是	调整森林检测时机

所有版本求的都是精确最优解；性能比较关注搜索工作量如何变化。

回溯中的候选集与高冲突节点

候选集 C：当前仍可讨论、且不与已选水果冲突的节点集合。



中心点 007 的度数最高

讨论 007 时

选 007：它和 5 个邻居都移出候选集

不选 007：只把 007 移出候选集

优先讨论高冲突节点，选中它时往往能一次缩小很多候选。

图中蓝色中心节点与 5 个叶节点冲突；每条边表示一条不能同时选取的规则。

两种选点策略：固定顺序与动态调整

静态预排序（v5）

回溯开始前，按每种水果的总冲突数从大到小排一次；拆分分量后，各分量沿固定顺序讨论。

优点：预处理简单；M ≤ 100，排序开销很小。

局限：顺序依据原始冲突数，不随候选集变化。

动态选点（v2）

每次回溯都在当前候选集中，选择与剩余候选冲突数最多的水果。

优点：选点反映当前真正剩下的冲突关系。

代价：每个搜索节点都要重新统计候选度数。

预期：简单图上静态顺序成本低；候选关系变化明显的复杂图上，动态顺序更能适应。

测试总体支持动态选点的预期

p95 求解时间（ms，越低越好）



超时次数 / 2460：　v2 = 0　　v3 = 9　　v5 = 4

banded M=100 中位数：v2 87.4 ms　•　v3 544.5 ms　•　v5 373.3 ms

三版均做分量分解；v2/v5 对比动态与静态选点，v3 使用固定下标顺序。

分量分解：孤立节点思路的延伸

孤立节点没有冲突边，可以直接选入；它其实是大小为 1 的连通分量。



不同连通分量之间没有边；一个分量的选取不会影响另一个分量。

分别求每个分量的局部最优解，再把这些解合起来：

$$S^* = S_1 \cup S_2 \cup \cdots \cup S_k$$

各分量得分可相加，所以合并后就是全图的最优解。

把“单个孤立水果可以先选”推广为“多个互不相连的子图可以分别求解”。

v2 与 v4：只差连通分量分解

两个版本都用动态最大度数选点；v4 不分解，v2 把各分量拆开搜索。

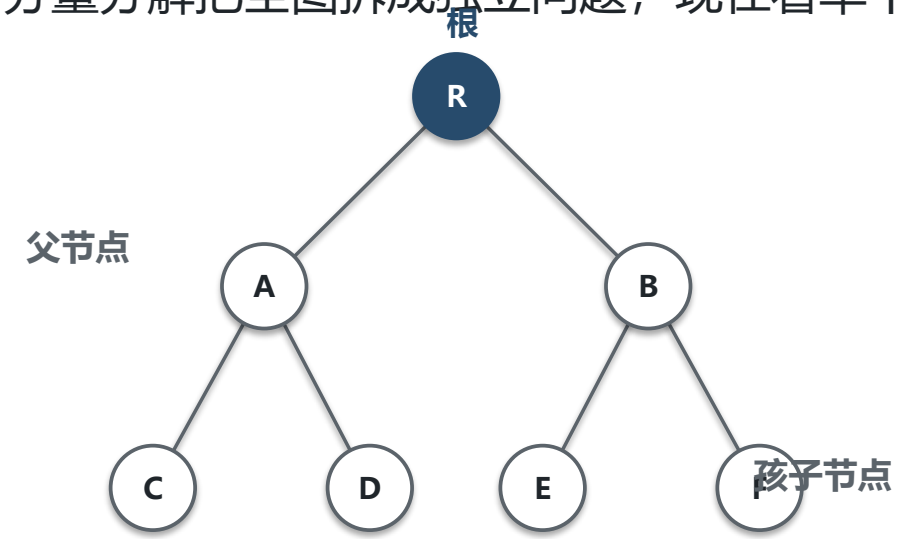
测试指标	v4：不分解	v2：分解	图形
components100 中位数	76.470 ms	0.083 ms	多分量
path100 中位数	149.884 ms	150.025 ms	单分量
全套 2460 组超时	21	0	full

多分量图收益显著；只有一个分量的路径图，拆分前后耗时接近。

来自 v1–v4 同轮测试：2460 组输入，5 秒超时上限；v2/v4 仅分解策略不同。

无环的连通分量就是树

分量分解把全图拆成独立问题；现在看单个连通分量有没有环。



无环的连通图就是树。

选一个根后，每个其他节点都有一个父节点，也可以有多个孩子。

选中父节点，孩子不能选；不选父节点，孩子可选也可不选。

不同孩子的子树之间没有边，可以分别计算再合并。

每个节点的选取状态只受父节点影响，也只影响子节点；这种局部传递很像马尔可夫链。

无环的连通图就是树；选定根后，状态按父子关系传递。

树上每个节点只需“选 / 不选”两个状态

take(v)：选中 v 时，v 的子树能得到的最高分；skip(v)：不选 v 时的最高分。

take(v) = w(v) + Σ skip(u)

u 是 v 的孩子：选 v 后，每个孩子都不能选。

skip(v) = Σ max(take(u), skip(u))

不选 v 后，每个孩子都可在选 / 不选中取分数较高的一种。

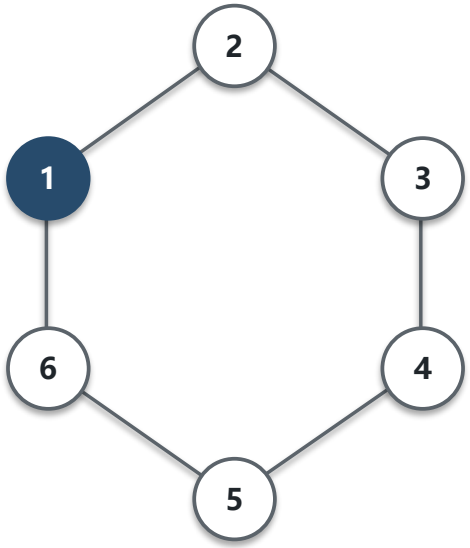
叶子：take(v)=w(v)，skip(v)=0；从叶子开始，一层层向根递推。

依据这两条递归关系，可以反向从叶子向根递推；最后从根回溯重建答案。

M=100 路径：v2 80.479 ms → v6 0.0046 ms；树：4.521 ms → 0.0071 ms。

v7：回溯后，环图可能变成森林

原始分量：有环



删去候选点

剩余候选图：无环



检测成功 → 森林 DP 结束该分支

67.204 → 0.0423 ms

100 点环图：v6 → v7

无环性由程序检查；检测失败时继续精确回溯。

v8：根据环秩调整森林检测时机

v7：固定阈值

初始环秩 < 3：删去 5 个候选点后首次检测
初始环秩 ≥ 3：剩余 ≤ 15 点时首次检测
失败后：再删去 5 个候选点才复查

v8：跟随初始环秩

r = 边数 − 点数 + 1

首次检测：删去 q = min(n, ⌈r/2⌉) 个候选点

失败后：一次分支使候选点减少 ≥ 2 时复查

环秩是打断所有环至少要删的边数；普通环上每个点连两条边，所以先从环秩的一半开始检测。这是经验起点。

取一个点可连同多个邻居一起移出候选集，因此 v8 倾向在这类分支后复查。

结构优化的整体效果

版本	p95（ms）	累计耗时（s）	100 点环图（ms）
v2 基线	25.145	16.721	69.597
v6 原始树 DP	10.509	11.041	67.204
v7 剩余森林 DP	2.294	1.612	0.0423
v8 检测时机调整	0.795	0.773	0.0147

测试集：2,460 组；四个版本均 0 超时。

局部反例：100 点核心外围图，v7 为 0.0546 ms，v8 为 0.0967 ms。

耗时为同一轮程序内部计时；小于 1 ms 的差值不宜单独解读。

结论

01	**选点**	动态度数降低困难实例的搜索尾部。
02	**分量**	拆开独立子图，避免组合搜索。
03	**结构**	树和回溯后形成的森林可由 DP 精确求解。

最终版本 v8：精确求解；在 2,460 组基准中全部完成并通过答案校验。

最终提交代码：v8_cycle_tree_dp.c；完整实验与报告见项目仓库。